



Steel Cloud Solutions

Reasoning Layer for Discovery

A proven AI stack, traced back to the original mandate

Period	Discovery front-office briefing
Author	Tobias Polly, polly consulting e.U.
Date	17 July 2026
Reference	SC-DISC-EXTRACT-2026-07
Status	Confidential

1 The mandate, and what it now does

The system in this briefing was commissioned in November 2025 under a single, narrow mandate (RFQ 125): build a **zero-data-leakage AI agent** that reads the *edit history* of enterprise transactions, tells a person in plain language which records are being changed and by whom, and does it inside Microsoft Teams – all within a locked-down network boundary so that no sensitive data leaves the customer’s control. That agent is live today. This extract strips the original design back to its load-bearing parts, drops the details specific to any one source system, and shows how the same engine points at Discovery’s own systems.

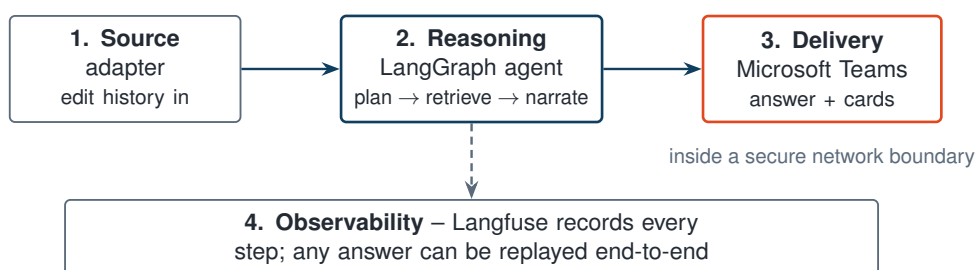
The first thing the agent was pointed at was booked orders. Every time an order is edited after it is committed – a price nudged down, a quantity trimmed, a discount reopened – the change is recorded but rarely read. Aggregated across thousands of records, those quiet edits are where booked revenue quietly leaks away. We now package this capability as the **Revenue Leakage Model**: it ranks the records, editors, and fields where post-commitment changes are eroding value, and narrates the pattern so a non-technical reader can act on it. The finance workstream is the live proof; the mechanism is general to *any* system where records are edited after a commitment is made.

What Discovery leaves with. The engine already runs in production on one workstream. Pointing it at a Discovery data source is a **configuration change, not a new build** – the reasoning, the audit trail, the Teams delivery, and the security posture are reused unchanged. That is the fast, low-risk path to a measurable P&L result.

In one paragraph: content enters through a source adapter, a bounded AI agent reasons over it and answers the user in Teams, and every answer is recorded end-to-end so a human can replay exactly how it was produced. The agent does not change between use cases – only the source in front of it and the questions asked of it do.

2 The architecture, stripped to four parts

SAP · custom systems · data lake



Four parts, each replaceable on its own:

- **Source adapter** – crosses the boundary from the customer’s system into the platform and presents edit history in a single, uniform shape. The source behind it is interchangeable (Section 5).
- **Reasoning** – a bounded AI agent that plans what to fetch, fetches it, and writes the answer in plain language. This is the part that is reused unchanged across every use case (Section 3).
- **Delivery** – Microsoft Teams. Users ask in plain English and get a written answer plus a structured card; no new tool to learn, no SQL, no dashboard.
- **Observability** – Langfuse sits underneath everything and records each step, so any answer can be reconstructed and defended after the fact.

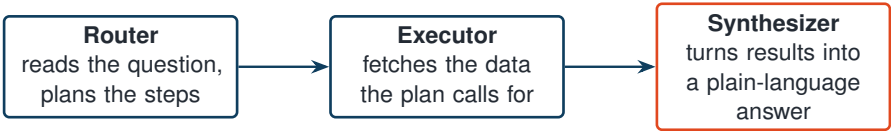
Standing the platform up for a new source reuses the top three parts wholesale; only the source, the storage, and the model deployment are set per engagement.

Component	For Discovery	What that involves
Reasoning agent	Live	Reused unchanged – the same bounded agent
Teams delivery	Live	Same bot, cards, and streaming
Audit trail (Langfuse)	Live	Same end-to-end replay of every answer
Security boundary	Live	Same closed-network, customer-tenant posture
Source adapter	Configured	Point it at the chosen live Discovery system
Compute layer	Configured	Stand up the ephemeral DuckDB / graph layer
Model	Configured	Confirm the model deployment in-tenant

Table 1: Four parts are reused as-is; three are configured. None is a new build.

3 How it reasons, and why every answer is auditable

The agent follows a fixed three-step path rather than being left to improvise. That choice is deliberate: a predictable path is cheaper to run, easier to budget, and – critically for regulated data – fully auditable.



Three steps, three model calls, one answer – bounded every time. A worked example on the Revenue Leakage workflow:

ANALYST ASKS

Where are we leaking the most revenue on last quarter’s orders?

ANSWER

Order 4500012345 was edited 12 times after booking – the most this quarter. Eight of those changes reduced the net price, all by two editors. Across the quarter, discount and net-price fields account for 71% of post-booking edits, concentrated in three accounts. Those three are where booked revenue is leaking fastest.

Two properties make this safe to put in front of sensitive data:

- **Bounded and auditable.** The plan and every data lookup are stored with the answer. There is no open-ended loop of tool calls, so cost and behaviour are predictable and reviewable.
- **Zero data leakage, fully replayable.** The agent runs inside a closed network boundary, and Langfuse records each step end-to-end. When someone asks “how do I know the AI didn’t make this up?”, the honest answer is: replay it and see.

3.1 Built to pass a security review

The stack is designed so that the security question is *finite*. When the platform reaches into a customer’s system, the registration lives in the **customer’s own tenant** – Steel Cloud holds only the operating secret and the discipline to rotate it, and the customer keeps the power to revoke access at any time. Data handled during a run is destroyed on a fixed schedule, the network defaults to deny, and every answer is logged for replay. For a business that has been burned by vendors before, the point is that nothing here requires trusting a black box: access is scoped, revocable, and observable end-to-end.

4 What we have already built, and what it taught us

None of this is a whiteboard proposal. The pieces below are running, and each one carries a lesson we paid for once so that Discovery does not have to. The payoff for Discovery is that you inherit the hardened version, not the first draft.

What we built	What it taught us
A bounded agent (plan → retrieve → narrate), tested on real data	Bounded beats clever: predictable cost, answers in a few seconds, and a plan you can audit – the reason it clears a compliance review where an open-ended agent would not.
A source adapter in front of the data	Swapping the data source is a configuration change, not a rebuild. This is the whole basis of the retargeting claim.
A business-language layer over the tools	Users ask in their own words, not the system’s; changing a tool never means rewriting the prompts.
Full replay of every answer (Langfuse)	The honest answer to “did the AI invent this?” is to replay it step by step – this is what earns trust with auditors and risk teams.
A human-approval gate on a finance automation	In regulated work, “the AI proposes and a person disposes” is what makes automation acceptable. The human stays in charge of anything that leaves the building.
One engine reused across several engagements	Roughly three-quarters of each new build is reuse; only the source, the measures, and the delivery cards change.

Table 2: The stack is a set of paid-for lessons, not a set of promises.

One lesson we are candid about: the first build kept the agent’s session memory in-process, which caps it to a single instance. The production path swaps in a shared store so it scales to many users at once – a small, known change made up front rather than discovered late. Naming the limits is part of how the platform stays trustworthy.

5 Where the data lives: query live, compute ephemerally

The platform does not keep a standing copy of the customer’s data. It queries the live system on demand, pulls back only what a question needs, and builds the analysis in an **ephemeral**

working layer that exists for the session and then disappears. That is both a simpler design and a stronger security posture: there is no second database of sensitive records to secure, reconcile, or destroy – because there is no second database.

5.1 Retrieve from the live system

The agent sees a uniform “edit history” interface; behind it, the adapter queries whichever live system the engagement runs on. SAP is one supported source, not a requirement.

Live source	Typical use	Status
SAP S/4 & DataSphere	Finance and order edit history	Live today
Custom / in-house systems	Discovery’s own matter and billing systems	Ready to point
Staffing & HR platforms	Bullhorn, Workday record changes	Planned
Data lake / warehouse	Consolidated cross-system history	Planned

Table 3: The adapter queries the live system. SAP is one card among several.

5.2 Compute in an ephemeral layer – DuckDB and graph

Once the data is retrieved, the analysis happens in a working layer that is spun up for the question and torn down after. Two engines cover the two kinds of question, and the platform uses either or both, as the table below sets out.

Working engine	Best at	Lifespan
DuckDB (tabular analytics)	Ranking, counting, trend and leakage summaries over large volumes; joining across sources; the fast slicing that follow-up questions need	Per session
Ephemeral graph	“How is this connected to that?” – tracing how edits, people, and matters relate across many hops	Per session
Persistent store (optional)	A standing, pre-computed view when a customer specifically wants one retained	Retained

Table 4: Analysis is built in an ephemeral layer and discarded; a persistent store is the exception, not the rule.

DuckDB is the general-purpose workhorse here. It handles large volumes of data locally and fast, which makes it as useful for matter and billing analysis, staffing, or cross-system reporting as it is for finance – the same engine, pointed at different data. The ephemeral graph is added on top only when the value is in the relationships rather than the totals. Neither leaves anything behind when the session ends.

6 The models, and what it takes to point at Discovery

The reasoning runs on models hosted in **Azure AI Foundry**, inside the same network boundary as the rest of the stack – so prompts and data never leave the customer’s Azure tenant. The

model is a configuration value, not a hard-wired dependency; it can be swapped as newer models arrive, or replaced with a fully self-hosted model where a customer requires it.

Option	Notes	Residency
Foundry – GPT-5.6 Sol	Current default; strongest structured reasoning	In-tenant
Foundry – Kimi K3	Alternate hosted model, same inter-face	In-tenant
Self-hosted (open model)	For a fully on-premises requirement	On-premises

Table 5: Models are swappable behind one interface. Nothing leaves the customer’s boundary.

6.1 Pointing the engine at Discovery

Because the source, the storage, and the model are all configuration, standing this up for Discovery is a short, low-risk sequence rather than a fresh build:

1. Point the source adapter at the chosen Discovery system’s live edit history.
2. Stand up the ephemeral DuckDB layer; add the graph only if relationship questions demand it.
3. Confirm the model deployment and the network boundary in Discovery’s tenant.
4. Validate the Revenue Leakage answers against a known period, then open it in Teams.

The position. The engine, the audit trail, the Teams experience, and the security posture are already built and running. Discovery does not fund another platform – it funds a retargeting. The fastest way to move the P&L is to reuse what already works.